CPMS 01

One

Britt Anderson

Winter 2025

Contents

1 Is Cognition Computation?

1.1 Some Additional Questions For Computational Cognition:

1. Is the brain a computer?
2. What does it mean for something to be computable?
3. What is the computational theory of mind, and what does it mean for us practically?

1.2 What Is Cognition?

The technical challenges for modeling in psychology are mastering the mathematics and programming necessary to formally express an idea and to implement a simulation to examine the consequences. The conceptual challenge is to decide what it is you are modeling. And it appears that there is no consensus on what is cognition.[1].

Which of these opinions, if any, do you agree with and why?

[1] Tim Bayne et al., "What Is Cognition?," *Current Biology* 29, no. 13 (2019): R608–15, https://doi.org/10.1016/j.cub.2019.05.044.

In the late 1800s Ebbinghaus, using creative, and for their time, innovative, Herculean methodologies, demonstrated that the proportion of items retained in memory declines predictably over time. The form of the forgetting curve is exponential.

Is a mathematical formula that reproduces an observed behavioral pattern a model?

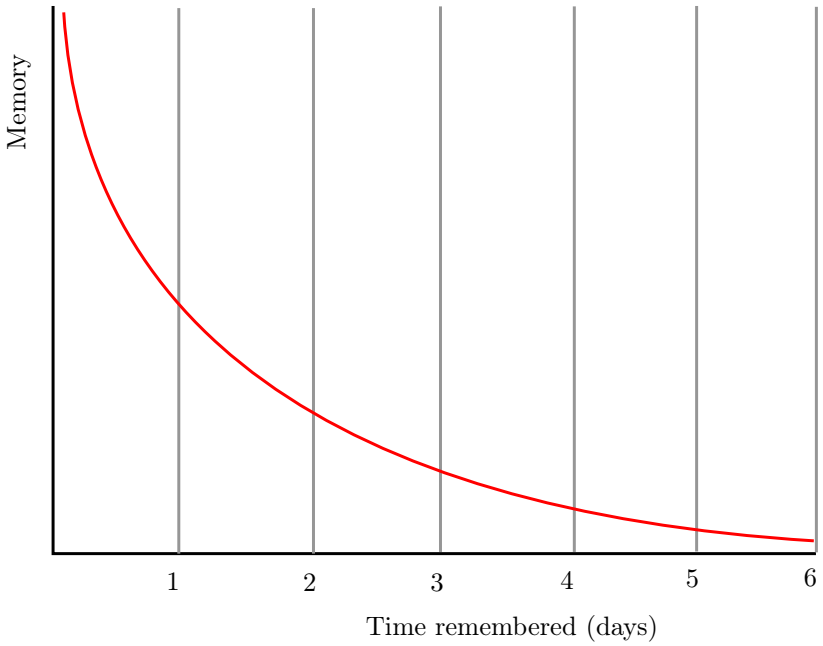

Figure 1: By The original uploader was Icez at English Wikipedia.

What can we infer from the similarity between Figure 2 and Figure 1?

1.3 Cognition is . . . ?

As revealed in[2] there are a wide array of views on what is cognition.

Which of these opinions, if any, do you agree with and why?

1.4 Is Cognition Computational?

1. Is there anything a human can think that a computer cannot compute?
2. What does it mean for a function to be "computable"?
3. What does it mean to say that cognition is computational?
4. What implications does the answer have for modeling cognition?

[2] Bayne et al.

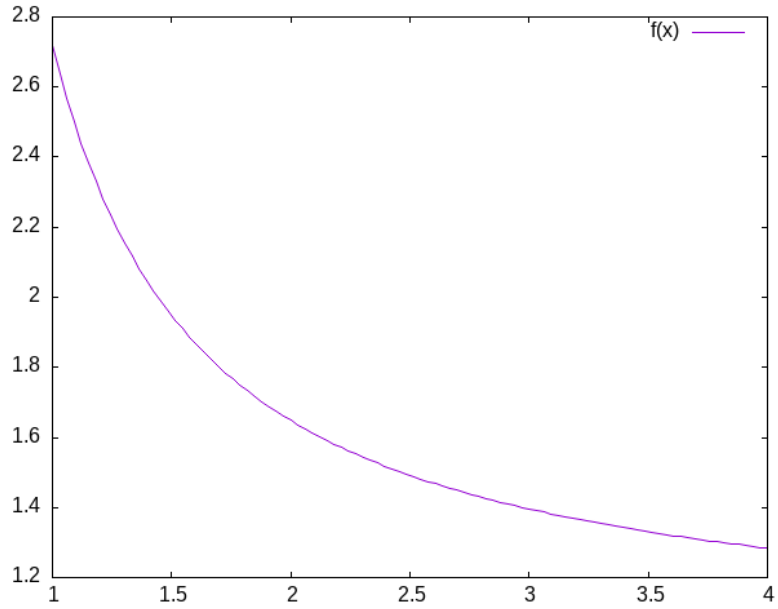

Figure 2: Plot of $e^{\frac{1}{x}}$.

1.4.1 Introduction

Is there a difference between computational cognitive neuroscience and cognitive computational neuroscience? The former seems to suggest that we are interested in explaining cognition directly from the actions of neurons and then using computational tools as adjuncts to aid this attempt. On the other hand, when you reverse the order it seems as if you are trying to explain thinking via the computational accounts of neuronal function. The latter seems to require a clear link between notions of computation and models of thinking. To use Marr's three levels as a scaffold[3]; we are taking neurons as our implementation, positing algorithms for them, and assuming that computational principles populate the level of our cognitive abstractions. Measures of success depend on the meanings of terms.

This reference is an older one, and not the original, but it does use LISP as its example, which is the classic example of good old fashioned AI. What are Marr's three levels?

Behind all of this is the idea that if you want to understand *mind* you need to first develop a *theory*. Explanations offered in words only are really more proto-theories than theories because of their imprecision. To express a theory clearly and unambiguously you need to *formalize* it. Either you express it as *math* or you can *code* it. Let's explores the distinction between computation as implementation and computation as the basis for mind.

1. Computing in Minds and Computers

[3]Ron McClamrock, "Marr's Three Levels: A Re-Evaluation," *Minds and Machines* 1, no. 2 (May 1991): 185–96, `https://doi.org/10.1007/bf00361036`.

One definition of whether something is computable is whether there exists a Turing Machine that can compute it. Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing[4, 5]

(a) What is a Turing machine? Some Background and Details Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:

   i. a finite alphabet,
   ii. a finite set of states,
   iii. the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and
   iv. a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.

   i. Classroom Activity Let's develop pseudo-code for the implementing a Turing Machine

(b) The Lambda Calculus

   Another model of computation is the *lambda calculus*[6].

   The lambda calculus was developed as a theory of functions. John McCarthy invented the computer programming language Lisp as a theoretical exercise for working on the theory of computable functions. He felt the Turing machine was too mechanical and too awkward for this sort of theoretical work. He wanted a better tool; a better metaphor. He adopted the lambda of the lambda calculus and the `eval` function to take in lisp programs and execute them. Later one of his collaborators observed that it was relatively straightforward to implement this theoretical language as a real programming language. A bit more of the history this development can be found here. To get more details see.[7]

   i. Some Details and Interesting Facts about the Lambda Calculus
      A. There is not just one lambda calculus.

[4] Alan Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)," in *The Essential Turing* (Oxford University PressOxford, 2004), 58–90, `https://doi.org/10.1093/oso/9780198250791.003.0005`.

[5] A nice short version of this history can be found in this pdf.

[6] And yet another is the *theory of recursive functions*

[7] Greg Michaelson see *An Introduction to Functional Programming through Lambda Calculus* (Mineola, N.Y: Dover Publications, 2011), `https://www.cs.rochester.edu/~brown/173/readings/LCBook.pdf`.

B. To have the lambda calculus you need to specify your algebra. What are the rules for reducing things (i.e. your computations), what are the allowed symbols, and what do things mean? A "lambda calculus" is a way of handling "lambda expressions."[8]

ii. Into the Weeds

Terms are either *simple variables* e.g. x or y or *composite terms* like $\lambda\ v\ t_1$. Having two terms next to each other $t_1\ t_2$ means "apply" t_1 to t_2. The meaning of a term like $\lambda\ v\ .\ t_1$ is the value returned by the lambda abstraction. The meaning part is sometimes designated in writing by formulas using arrows, such as $t_1\ \rightarrow\ t_2$.[9]

Functions have the form λ <name> . <body> . Note the "dot". This separates the name from the body of expressions that it names. <body> is also an expression (note the recursion that is built in).

What are recursive functions?

Some "axioms" you supply, or that Alonzo Church did are:

- Beta reduction apply the function, e.g. (λ x.M) N = M[x:N] this latter is just a way of saying that you will substitute the value "N" everytime you see an "x" in whatever expression "M" is.
- Alpha conversion essentially you are just renaming variables
- Eta reduction Is a way of getting rid of things that don't change anything. If for example $f(x) = g(x)$ then you could just save yourself some ink and write: f = g. A more technical way of saying things, that we will not go into, is that you can free yourself of free variables, that is if there is no "x" in "f" then $f = \lambda x.(fx)$.

Your goal is often the **normal** form. A lambda term that cannot be reduced anymore. Think of it like simplifying an equation. It helps you get to the essentials, and can make it easier to compare things to see when they are the same.

The equivalent of the halting problem for the Turing machine is the reaching of a *normal form* in the lambda calculus.

iii. Some Opportunities for Practice

- Write the lambda expression for the identity function?
  - But first we better decide what the identity function is?
- Apply the identity function to itself.
- What is the identity function in whatever language you are hoping to use?

[8]The λ of lambda calculus doesn't really mean anything.It just signals that you have a lambda expression. Its creation as the symbol for the function calculus was an accident of notation and the limits of older typewriters.

[9]Want to try implementing the lambda calculus yourself? Here is a blog post on doing this in the computer language Common Lisp.

- An interesting lambda expression is the "self-application" expression: $\lambda\ s.(s\ s)$.
  - With pencil and paper try to:
    - apply this to the identity,
    - apply the identity to the self-application,
    - apply the self-application to itself. What is its termination status?

(c) What are the differences? We have seen Turing Machines and a Lambda Calculus. They look very different. Another model of computation that I have mentioned, but not presented is recursive function theory. It looks different still.

Class Discussion

What does it mean for our idea that the mind is a "computer" if there are three different models of computation?

Always question assumptions: Church-Turing Conjecture .

Further Class Discussion

1. Are Turing machines /digital?
2. Is this an important distinction?
3. Does this mean that analog computations are omitted?
4. Would an analog paradigm be a better match to modeling mental activity?

2. Classical Computational Theory of Mind

Class Discussion

Does a computational theory of mind mean that minds are multiply realizable ?

The classical computational theory of mind says that in all important ways the mind is like a Turing machine. If we want to model (or emulate) functions of mind one way would be to build a Turing machine. However, this can be a practical challenge, and one can question the insight gained from this approach. Still given the theoretical and practical prominence given to Turing computability it behooves us to know what a Turing machine truly is.

For something to be computatble the Turing machine that computes it must halt. So, we would like to know before we run our machine whether it will halt or not. Unfortunately, the answer to the halting problem is that we cannot know in advance, and in general, whether a Turing machine will halt. Computabilty is therefore *undecidable*.

3. Programming a Turing Machine: the Busy Beaver

   In my experience the only way to really come to grips with what a Turing machine is is to program one yourself. For this exercise we will program a Turing machine to solve an elementary version of the Busy Beaver problem. This is a famous problem, because it is easy to state and woefully difficulty to answer. In fact it is incomputable.

   The following is a slightly formatted version of the Wikipedia description of a Turing machine.

   (a) A Turing machine has n "operational" states plus a Halt state, where n is a positive integer, and one of the n states is distinguished as the starting state.

   (b) The machine uses a single two-way infinite (or unbounded) tape.

   (c) The tape alphabet is {0, 1}, with 0 serving as the blank symbol.

   (d) The machine's transition function takes two inputs:

      i. the current non-Halt state,

      ii. the symbol in the current tape cell,

      iii. and produces three outputs:

         A. a symbol to write over the symbol in the current tape cell (it may be the same symbol as the symbol overwritten),

         B. a direction to move (left or right@";" that is, shift to the tape cell one place to the left or right of the current cell), and

         C. a state to transition into (which may be the Halt state).

   We will use it to guide us to write a simple instance that computes a solution to the Busy Beaver problem. A n-state Turing machine has $(4n + 4)2n$states. The formula is (symbols $\times$ directions $\times$ (states + 1))(symbols $\times$ states). The transition function (how to figure out where to go next may be seen as a finite look-up table. Each row of the table is a 5-tuple: (current state, current symbol, symbol to write, direction of shift, next state). Our goal with the Busy Beaver problem is to run our machine to produce as long a series of uninterrupted ones as we can and then **halt**.

   What it means to "run" a Turing machine is to start in the starting state with the current tape cell being any cell of a blank (all-0) "tape", and then iterate the transition function. If the Halt state is entered then the number of 1s remaining on the tape is called the machine's score. Different transition rules will give us different outputs, so we can score our machine based on its performance.

   To restate in a more general way, the n-state busy beaver (BB-n) game is a contest to find an n-state Turing machine having the largest possible score — the largest number of 1s on its tape after halting. A machine that attains the largest possible score among all n-state Turing machines

is called an n-state busy beaver, and a machine whose score is merely the highest so far attained (perhaps not the largest possible) is called a champion n-state machine.[10]

(a) A Busy Beaver Warm-Up

A simple version of the Busy Beaver problem, and one you can do by hand with pencil and paper, is the n=2 version. Create a Turing Machine with the following transition rules:

Here is the transition table for our machine:

Machine Tuple In	Machine Tuple Out
a0	b1r
a1	b1l
b0	a1l
b1	h1r

(b) Busy Beaver Problem Demo Code in Racket Why Racket? Because I don't want you to copy my code. I want you to see the structure of how I approach the problem with this language so you can build your own version in another language.

```
(struct turing-machine (state tape head-location) #:transparent #:mutable)]
```

Figure 3: Making a Structure for Our Turing Machine

```
(define (move-left temp-tm )
   (let ([loc (turing-machine-head-location temp-tm)]
         [lst (turing-machine-tape temp-tm)])
      (if (= loc 0)
          (set-turing-machine-tape! temp-tm (cons 0 lst))
          (set-turing-machine-head-location! temp-tm (- loc 1)))))

(define (move-right temp-tm )
   (let ([loc (turing-machine-head-location temp-tm)]
         [lst (turing-machine-tape temp-tm)])
     (when (= (+ loc 1) (length lst))
       (set-turing-machine-tape! temp-tm (append lst (list 0))))
     (set-turing-machine-head-location! temp-tm (+ loc 1))))

(define (move tm dir)
  (cond
    [(equal? dir 'left) (move-left tm)]
    [(equal? dir 'right) (move-right tm)]
    [else (error "illegal direction")]))
```

Figure 4: Moving our TM Along the Tape

[10] Much of this description of the Busy Beaver problem is lightly edited Wikipedia material.

```
(define (tm-equal-state-value tm state value)
  (and (equal? (turing-machine-state tm) state)
       (= (list-ref (turing-machine-tape tm) (turing-machine-head-location tm)) val

(define (upd-tape-location tm value)
  (set-turing-machine-tape! tm (list-set (turing-machine-tape tm) (turing-machine-h

(define (pretty-print-tm tm)
  (display (format "state:~a, tape: ~a, head: ~a\n" (turing-machine-state tm) (turi
```

Figure 5: Helper Functions

```
(define (rule tm) ;;state value
    (cond
      ((tm-equal-state-value tm 'a 0)
       (set-turing-machine-state! tm 'b)
       (upd-tape-location tm 1)
       (move tm 'right))
      ((tm-equal-state-value tm 'a 1)
       (set-turing-machine-state! tm 'b)
       (upd-tape-location tm 1)
       (move tm 'left))
      ((tm-equal-state-value tm 'b 0)
       (set-turing-machine-state! tm 'a)
       (upd-tape-location tm 1)
       (move tm 'left))
      ((tm-equal-state-value tm 'b 1)
       (set-turing-machine-state! tm 'h)
       (upd-tape-location tm 1)
       (move tm 'right))))
```

Figure 6: Rules are the Critical Part of This Machine

```
(define (busy-beaver-2-do tm)
  (pretty-print-tm tm)
  (do ([i 0 (+ i 1)])
    ((equal? (turing-machine-state tm) 'h)
     (display (format "Loops equaled ~a\n" i)))
    (rule tm)
    (pretty-print-tm tm))
  tm)
```

Figure 7: An n=2 run through

```
(begin
  (require "./code/tm.rkt")
```

```
(define initial-turing-machine (turing-machine 'a (list 0) 0))
(define working-tm (struct-copy turing-machine initial-turing-machine))
(busy-beaver-2-do initial-turing-machine))]
```

Figure 8: Testing The TM Busy Beaver

1.4.2 Busy Beaver Homework

See Dropbox on Learn

You might find the racket source useful for hints.

1.5 Planning for next time

See if you can get a sense of what *functionalism* means.